

# Priority List Linked List

## Pattern 1 — Two Pointer (Fast & Slow) ★★★★★

### What should you know?

- Finding middle node
- Detect cycle
- Find cycle start
- Happy number
- Kth node from end
- Remove nth node from end

### Interview Concepts

Fast pointer moves **2 steps**

Slow pointer moves **1 step**

Eventually

- Slow reaches middle
- Fast detects cycle

Time: **O(n)**

Space: **O(1)**

---

### Must-do Problems

#### Easy

- Middle of Linked List
- Linked List Cycle

#### Medium

- Linked List Cycle II
- Remove Nth Node From End
- Reorder List
- Delete Middle Node
- Rotate List

#### Hard

- Split Circular Linked List
- Find Loop Length

---

## Pattern 2 — Reversal Pattern ★★★★★

Almost every company asks one reversal question.

Know

- Reverse complete list
- Reverse first k nodes
- Reverse between left and right
- Reverse in groups
- Reverse recursively

---

### Must-do Problems

#### Easy

- Reverse Linked List

#### Medium

- Reverse Linked List II
- Reverse Nodes in K Group
- Swap Nodes in Pairs
- Reverse Even Length Groups

#### Hard

- Reverse Alternate K Nodes
-

## Pattern 3 — Merge Pattern ★★★★★

Know

- Merge two sorted lists
- Merge K lists
- Merge while maintaining order

### Must-do Problems

Easy

- Merge Two Sorted Lists

Medium

- Merge In Between Linked Lists
- Sort List

Hard

- Merge K Sorted Lists

## Pattern 4 — Dummy Node Technique ★★★★★

One of the most important interview tricks.

Instead of handling

```
head changes
first node
empty list
```

Create

```
dummy -> head
```

Now manipulate everything after dummy.

Used in nearly every Linked List problem.

### Must-do Problems

- Remove Elements
- Remove Duplicates
- Swap Nodes
- Partition List
- Merge Two Lists
- Remove Nth Node
- Odd Even Linked List

## Pattern 5 — Cycle Detection ★★★★★☆

Know

- Floyd Algorithm
- Detect cycle
- Find cycle entry
- Length of cycle

### Must-do Problems

- Linked List Cycle
- Linked List Cycle II
- Happy Number
- Circular Array Loop

## Pattern 6 — Partition / Rearrangement ★★★★★☆

Very common in Amazon & Meta.

Know

Move nodes

instead of swapping values.

---

### Must-do Problems

- Partition List
- Odd Even Linked List
- Reorder List
- Rotate List
- Segregate Even Odd Nodes
- Delete Duplicates

---

### Pattern 7 — Mathematical / Carry Pattern ★★☆☆☆

Whenever numbers are stored inside LL.

Know

Carry propagation

Borrow

Digit addition

---

### Must-do Problems

- Add Two Numbers
- Add Two Numbers II
- Plus One Linked List
- Double a Number Represented as LL

---

### Pattern 8 — Palindrome Pattern ★★☆☆☆

Uses

- Fast Slow
- Reverse second half

---

### Must-do Problems

- Palindrome Linked List
- Maximum Twin Sum

---

### Pattern 9 — Intersection Pattern ★★☆☆☆

Classic interview question.

Know

Two pointers switching heads.

```
A -> B
```

```
B -> A
```

Eventually they meet.

---

### Must-do Problems

- Intersection of Two Linked Lists

---

### Pattern 10 — Clone Random Pointer ★★☆☆☆

Very famous Google question.

Need

```
next pointer  
random pointer
```

Techniques

- HashMap

- O(1) interleaving

---

### Must-do Problems

- Copy List with Random Pointer

---

## Pattern 11 — Recursion ★ ★ ☆ ☆ ☆

Mostly tests recursion understanding.

Know

Reverse recursively

Delete recursively

Flatten recursively

---

### Must-do Problems

- Reverse Linked List (Recursive)
- Swap Pairs Recursive

---

## Pattern 12 — Basic Traversal ★ ★ ☆ ☆ ☆

These are building blocks.

Know

- Search
- Insert
- Delete
- Length
- Print
- Find tail

Not directly asked as standalone interview questions.

---

## The 20 Most Important Linked List Problems

These cover almost every pattern and are excellent interview practice:

| Problem                           | Pattern             | Difficulty |
|-----------------------------------|---------------------|------------|
| Reverse Linked List               | Reversal            | Easy       |
| Middle of Linked List             | Fast Slow           | Easy       |
| Linked List Cycle                 | Fast Slow           | Easy       |
| Merge Two Sorted Lists            | Merge               | Easy       |
| Remove Nth Node From End          | Fast Slow + Dummy   | Medium     |
| Reverse Linked List II            | Reversal            | Medium     |
| Reverse Nodes in K Group          | Reversal            | Hard       |
| Linked List Cycle II              | Cycle               | Medium     |
| Palindrome Linked List            | Palindrome          | Easy       |
| Reorder List                      | Fast Slow + Reverse | Medium     |
| Partition List                    | Partition           | Medium     |
| Odd Even Linked List              | Partition           | Medium     |
| Rotate List                       | Partition           | Medium     |
| Add Two Numbers                   | Carry               | Medium     |
| Add Two Numbers II                | Carry               | Medium     |
| Intersection of Two Linked Lists  | Intersection        | Easy       |
| Merge K Sorted Lists              | Merge               | Hard       |
| Copy List with Random Pointer     | Clone               | Medium     |
| Sort List                         | Merge Sort          | Medium     |
| LRU Cache (Linked List + HashMap) | Dummy Node          | Hard       |

## 10 Hidden Tricks Interviewers Expect You to Know

These are reusable techniques that often unlock multiple problems:

1. **Dummy Node** – Simplifies insertions/deletions at the head.
2. **Fast & Slow Pointers** – Find middle, detect cycles, split lists.
3. **Three-Pointer Reversal** – `prev`, `curr`, `next`.
4. **Tail Pointer** – Efficiently build new lists during partitioning/merging.
5. **Head Insertion** – Insert nodes at the front while reversing or cloning.
6. **Two-Pointer Head Switching** – Elegant solution for intersection detection.
7. **Reverse the Second Half** – Used in palindrome and reorder list problems.
8. **Split → Process → Merge** – Common in reorder, merge sort, and other divide-and-conquer solutions.

9. **HashMap + Linked List** – Essential for LRU Cache and random pointer cloning.

10.**In-place Node Manipulation** – Prefer changing pointers over swapping node values.

---

### If you're preparing for SWE interviews

I'd recommend mastering the patterns in this order:

1. Two Pointer (Fast & Slow)
2. Reversal Pattern
3. Dummy Node Technique
4. Merge Pattern
5. Cycle Detection
6. Partition / Rearrangement
7. Mathematical / Carry
8. Palindrome
9. Intersection
- 10.Clone Random Pointer
- 11.Recursion
- 12.Basic Traversal

Mastering the first **7 patterns** and solving the **20 problems** listed above will prepare you for the vast majority of Linked List interview questions asked in software engineering interviews.

---

---